

Robust Model Predictive Control

Learning Dynamical Systems

Lorenzo Favaron
2128830

Luca Galeazzo
2158514

Lorenzo Trubian
2125512

July 29, 2026

1 Introduction

Given the system with state $x \in \mathbb{R}^n$, output $y \in \mathbb{R}^p$, input $u \in \mathbb{R}^m$, white Gaussian noise with unit variance $v \in \mathbb{R}^{n+p}$

$$\begin{aligned}x_{t+1} &= Ax_t + Bv_t + Ku_t \\ y_t &= Cx_t + Dv_t\end{aligned}$$

defined by matrices: state-state matrix $A \in \mathbb{R}^{n \times n}$, noise-state matrix $B \in \mathbb{R}^{n \times (n+p)}$, state-output matrix $C \in \mathbb{R}^{p \times n}$, noise-output matrix $D \in \mathbb{R}^{p \times (n+p)}$, and input-state matrix $K \in \mathbb{R}^{n \times m}$.

A common assumption, also made in Section III of [1], is that the matrix $\begin{bmatrix} B \\ D \end{bmatrix}$ is square and invertible.

2 Model Predictive Control

$$\begin{aligned}x_{t+1} &= A_t x_t + K u_t \\ y_t &= C_t x_t\end{aligned}$$

over the time horizon $[t, t + N]$, in the presence of polyhedral constraints on the state and input variables. Let the control sequence be $\mathbf{u}_t = (u_t, u_{t+1}, \dots, u_{t+N-1})$. The cost function is

$$J(x_t, \mathbf{u}_t, t) = \sum_{i=0}^{N-1} [\|y_{t+i} - r_{t+i}\|_Q + \|u_{t+i} - \bar{u}_{t+i}\|_R] + \|y_{t+N} - r_{t+N}\|_{P_f} \quad (1)$$

This expression can be written w.r.t. the states x

$$J(x_t, \mathbf{u}_t, t) = \sum_{i=0}^{N-1} [\|C_t x_{t+i} - r_{t+i}\|_Q + \|u_{t+i} - \bar{u}_{t+i}\|_R] + \|C x_{t+N} - r_{t+N}\|_{P_f}.$$

It possible to isolate the quadratic and linear terms in the variables x and u

$$\begin{aligned}
J(x_t, \mathbf{u}_t, t) &= \sum_{i=0}^{N-1} \left[\|x_{t+i}\|_{C_t^\top Q C_t} + \|u_{t+i}\|_R \right] + \|x_{t+N}\|_{C_t^\top P_f C_t} \\
&\quad - 2 \sum_{i=0}^{N-1} \left[(C_t^\top Q r_{t+i})^\top x_{t+i} + \bar{u}_{t+i}^\top R u_{t+i} \right] + (C_t^\top P_f r_{t+N})^\top x_{t+N} \\
&\quad + \sum_{i=0}^{N-1} [\|r_{t+i}\|_Q + \|\bar{u}_{t+i}\|_R] + \|r_{t+N}\|_{P_f}
\end{aligned}$$

where the last term of the latter expression is a constant term and can be ignored in the optimization process. Let us define

$$z = \begin{bmatrix} x_{t+1} \\ \vdots \\ x_{t+N} \\ u_t \\ \vdots \\ u_{t-1+N} \end{bmatrix}, \quad r_{x,u} = \begin{bmatrix} r_{t+1} \\ \vdots \\ r_{t+N} \\ \bar{u}_t \\ \vdots \\ \bar{u}_{t-1+N} \end{bmatrix}$$

The *non-condensed form* of the MPC problem can be written as¹

$$\begin{aligned}
\min_z J(x_t, \mathbf{u}_t, t) &= \min_z \frac{1}{2} z^\top \bar{H} z + f^\top z \\
\text{subject to} \quad &G_{eq} z = x_{t+1,t+N}.
\end{aligned} \tag{2}$$

where the system evolution is encoded as equality constraint using the matrices

$$G_{eq} = \left[\begin{array}{cccc|cc} I & & & & -K & \\ -A_2 & I & & & -K & \\ & -A_3 & I & & & -K \\ & & \ddots & \ddots & & \ddots \\ & & & -A_N & I & -K \end{array} \right] x_{t+1,t+N} = \begin{bmatrix} \hat{x}_{t+1,free} \\ 0 \\ \vdots \\ 0 \end{bmatrix}. \tag{3}$$

Let us define the following matrices

$$\bar{C} = \left[\begin{array}{ccc|ccc} C_1 & & & & & \\ & \ddots & & & & \\ & & C_N & & & \\ \hline & & & I & & \\ & & & & \ddots & \\ & & & & & I \end{array} \right], \quad H = \left[\begin{array}{ccc|ccc} Q & & & & & \\ & \ddots & & & & \\ & & P_f & & & \\ \hline & & & R & & \\ & & & & \ddots & \\ & & & & & R \end{array} \right] \tag{4}$$

which allows to compactly write the cost function of (1)

$$\bar{H} = \bar{C}^\top H \bar{C}, \quad f = -\bar{C}^\top H r_{x,u}.$$

¹This formulation of the cost function is consistent with the standard form employed by quadratic programming solvers, such as MATLAB's *quadprog* and OSQP.

A natural question noticing the form of (3) is: what role do $x(k)$ and E_{eq} actually play in this problem? The key point is that the starting condition is not simply the current state, but rather the predicted free evolution based on the current state. This becomes especially important when estimators are involved: it is not strictly necessary to provide the MPC with x_t ; instead, one can supply the one-step-ahead prediction $\hat{x}_{t+1,free} = \hat{x}_{free}(t+1 | t)$ of the free evolution (the dependence on the information available at time t is omitted, since this is the most recent information accessible to the controller and thus the most accurate). In practice, this is what we have done, and the matrix E_{eq} is completely discarded, with the state-evolution constraints being reformulated accordingly as

$$G_{eq}z = x_{t+1,t+N} \quad \text{with} \quad x_{t+1,t+N} = \begin{bmatrix} \hat{x}_{t+1,free} \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

2.1 RKF approach

Even though we are working with time invariant system, we present the more general time-varying formulation because the least-favorable model has this property. Therefore, in the following it will be sufficient to substitute A_t with A , and C_t with C for all t when considering the RKF approach (that is, only the prediction of the free evolution is passed to the MPC solver) and A_t with $\tilde{A}_t$, and C_t with $\tilde{C}_t$ in the LFM approach (that is when the least-favorable model associated with the tolerance c is considered the nominal model by the MPC) Flowcharts 3c and 3d. Algorithm 3

2.2 LFM approach

Flowcharts 3a and 3b. Algorithms 1 and 2

3 Selection of the optimal tolerance c

Up to this point, the tolerance c has been chosen a priori. The proposed approach instead allows the controller to determine c by adding an additional term to the quadratic programming (QP) problem (2) with an additional term. In this formulation, the controller autonomously balances the quality of the prediction achieved so far against the prediction adopted by the optimal control law.

$$\min_{c,u,\hat{y},\hat{x}} \sum_{t=-L}^0 \beta^t \|y_t - \hat{y}_{t|t-1}(c)\|^2 + J(x_t, \mathbf{u}_t, t) \quad (5)$$

As illustrated by the global scheme in Figure 1, the minimization problem (5) is addressed only for a discrete set of tolerance values c . The main difficulty in tackling the full minimization problem in a single step lies in the fact that there is no closed-form expression for the Lagrange multiplier λ that satisfies equation

$$\gamma_t(\lambda_t) = \frac{1}{2} \left[\text{tr} \left((I_n - \lambda_t^{-1} P_{t+1})^{-1} - I_n \right) + \ln \det (I_n - \lambda_t^{-1} P_{t+1}) \right] = c \quad (6)$$

3.1 Best prediction strategy

Once the optimal tolerance has been selected in the MPCs, using the free-evolution prediction $\hat{x}_{c_j,free}(k+1|k)$ obtained from the RKFs, the optimal prediction $\hat{x}_{c_{best}}(k+1|k)$ (which already

includes both the free evolution and the effect of the selected input) will serve as the initial condition for all filters in the subsequent iteration. 1 3b 3d

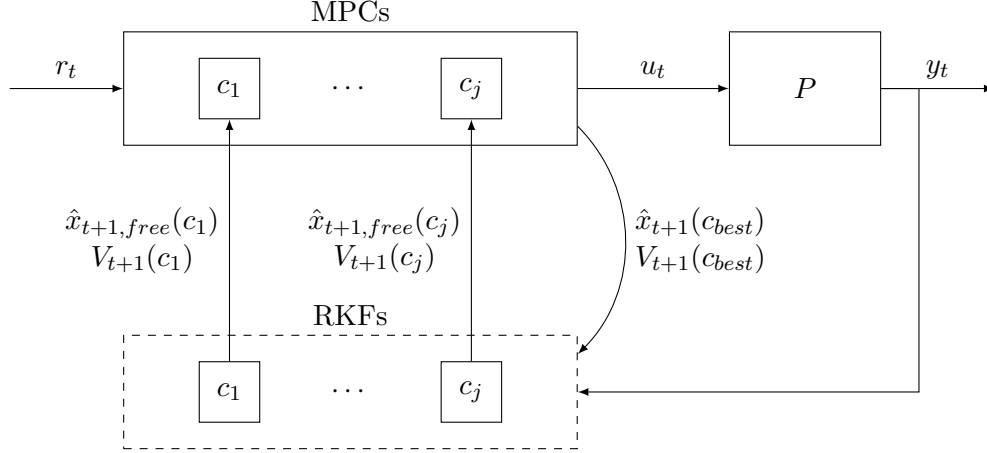


Figure 1: The dashed line emphasizes that each RKF operates independently from the others (apart from the shared initial estimate $\hat{x}$ and the measured output y), and therefore no computations are performed across filters. This independence does not hold for the MPCs: after each MPC solves its own optimization problem, an additional minimization problem is formulated that also incorporates past predictions, producing a single control input u along with an optimal tolerance and the corresponding best prediction $\hat{x}_{best}$.

3.2 Own prediction strategy

3 2 3a 3c

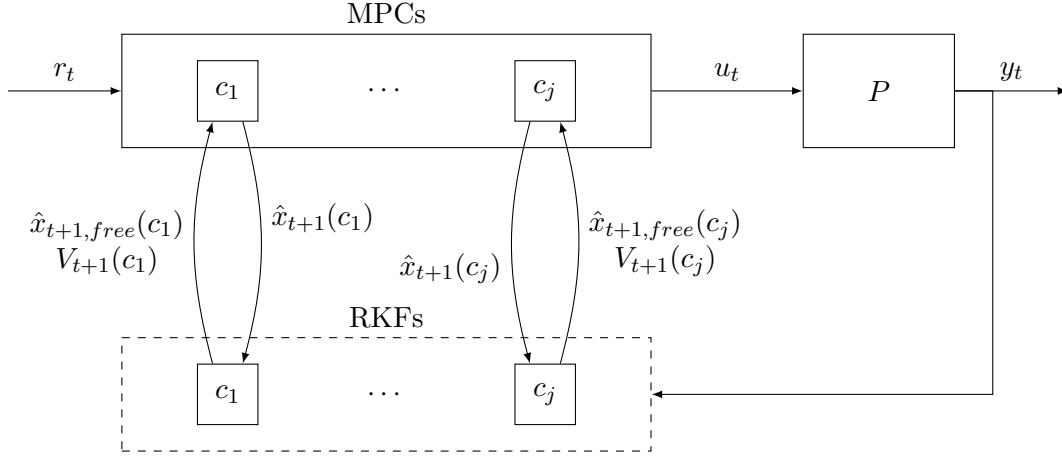
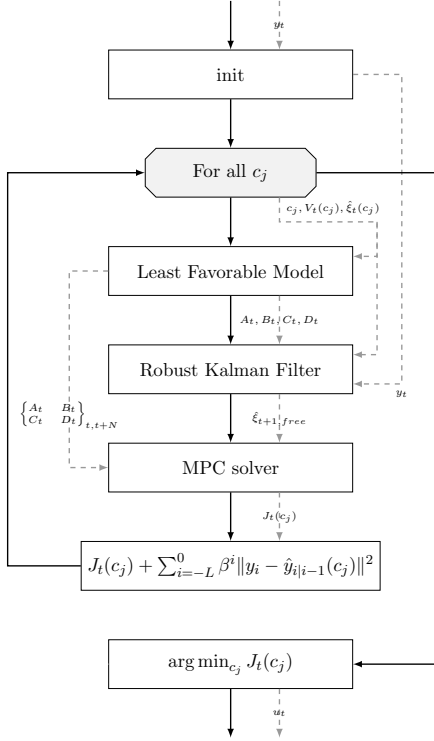


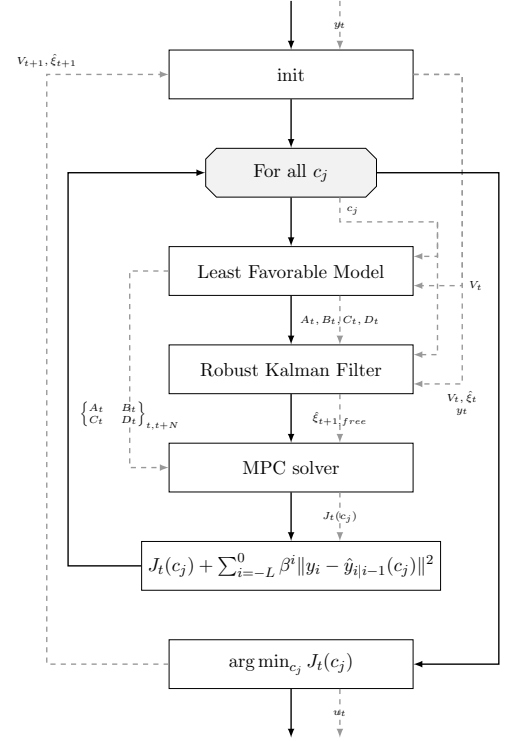
Figure 2

Appendices

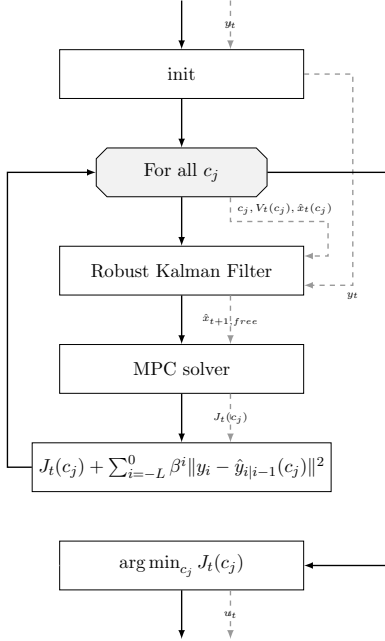
A Flowcharts



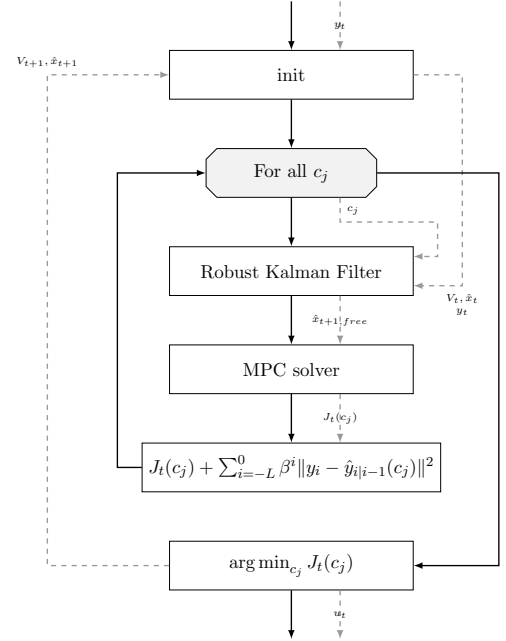
(a) LFM approach, own prediction strategy



(b) LFM approach, best prediction strategy



(c) RKF approach, own prediction strategy



(d) RKF approach, best prediction strategy. It is not different from using a single tolerance c .

Figure 3: LFM approach, best prediction strategy

B Pseudo-code

Algorithm 1 Robust MPC with tolerance selection at time t (LFM + best prediction).

Require: $\{c_j\}_j, \hat{\xi}_t, y_t, V_t, N, \beta, L$

Ensure: $u_t, \hat{\xi}_{t+1}, V_{t+1}$

```

1: for all  $c_j$  do
2:    $\triangleright$  Algorithm 5 to compute LFM in time interval  $[t, t + N]$ 
3:    $\{\tilde{A}_i, \tilde{B}_i, \tilde{C}_i, \tilde{D}_i\}_{i \in [t, t+N]} \leftarrow \text{LFM}(c, V_t, N)$ 
4:    $\triangleright$  Algorithm 4 applied using the best information of the previous step
5:    $\hat{\xi}_{t+1, \text{free}}(c_j), V_{t+1}(c_j) \leftarrow \text{RKf}(c_j, \hat{\xi}_t, y_t, V_t, \{\tilde{A}_t, \tilde{B}_t, \tilde{C}_t, \tilde{D}_t\})$ 
6:    $\triangleright$  MPC solver with LFM as nominal model
7:    $u_t(c_j), \hat{\xi}_{t+1}(c_j), J_t(c_j) \leftarrow \text{MPC}(\hat{\xi}_{t+1, \text{free}}(c_j), \{\tilde{A}_i, \tilde{B}_i, \tilde{C}_i, \tilde{D}_i\}_{i \in [t, t+N]}, N)$ 
8:    $\triangleright$  Add past prediction errors to the controller cost
9:    $J_t(c_j) \leftarrow J_t(c_j) + \sum_{i=-L}^0 \beta^i \|y_i - \hat{y}_{i|t-1}(c_j)\|^2$ 
10:  $\triangleright$  Optimization step, problem (5)
11:  $c_{\text{best}} \leftarrow \arg \min_{c_j} J_t(c_j)$ 
12:  $\triangleright$  Update prediction and covariance for the next step
13:  $\hat{\xi}_{t+1} \leftarrow \hat{\xi}_{t, \text{free}}(c_{\text{best}}) + \begin{bmatrix} K \\ 0 \end{bmatrix} u_t(c_{\text{best}})$ 
14:  $V_{t+1} \leftarrow V_{t+1}(c_{\text{best}})$ 
15:  $\triangleright$  Apply input
16:  $u_t \leftarrow u_t(c_{\text{best}})$ 

```

It is worth noting that step 13 of Algorithm 1 is actually redundant, since the solution z of the minimization problem (2) already includes the prediction of the next state $\hat{x}_{t+1}$. It is presented explicitly only to illustrate as clearly as possible how the next prediction is a combination of the Robust Kalman Filter and the Model Predictive Controller.

Algorithm 2 Robust MPC with tolerance selection at time t (LFM + own prediction).

Require: $\{c_j\}_j, y_t, N, \beta, L$

Ensure: u_t

```

1: for all  $c_j$  do
2:    $\triangleright$  Algorithm 5 to compute LFM in time interval  $[t, t + N]$ 
3:    $\{\tilde{A}_i, \tilde{B}_i, \tilde{C}_i, \tilde{D}_i\}_{i \in [t, t+N]} \leftarrow \text{LFM}(c, V_t(c_j), N)$ 
4:    $\triangleright$  Algorithm 4 applied using the information of the previous step of the filter associated to
      the same  $c_j$ 
5:    $\hat{\xi}_{t+1, \text{free}}(c_j), V_{t+1}(c_j) \leftarrow \text{RKf}(c_j, \hat{\xi}_t(c_j), y_t, V_t(c_j), \{\tilde{A}_t, \tilde{B}_t, \tilde{C}_t, \tilde{D}_t\})$ 
6:    $\triangleright$  MPC solver with LFM as nominal model
7:    $u_t(c_j), \hat{\xi}_{t+1}(c_j), J_t(c_j) \leftarrow \text{MPC}(\hat{\xi}_{t+1, \text{free}}(c_j), \{\tilde{A}_i, \tilde{B}_i, \tilde{C}_i, \tilde{D}_i\}_{i \in [t, t+N]}, N)$ 
8:    $\triangleright$  Add past prediction errors to the controller cost
9:    $J_t(c_j) \leftarrow J_t(c_j) + \sum_{i=-L}^0 \beta^i \|y_i - \hat{y}_{i|i-1}(c_j)\|^2$ 
10:  $\triangleright$  Optimization step, problem (5)
11:  $c_{\text{best}} \leftarrow \arg \min_{c_j} J_t(c_j)$ 
12:  $\triangleright$  Apply input
13:  $u_t \leftarrow u_t(c_{\text{best}})$ 

```

Algorithm 3 Robust MPC with tolerance selection at time t (RKf + own prediction).

Require: $\{c_j\}_j, y_t, N, \beta, L$

Ensure: u_t

```

1: for all  $c_j$  do
2:    $\triangleright$  Algorithm 4 applied using the information of the previous step of the filter associated to
      the same  $c_j$ 
3:    $\hat{\xi}_{t+1, \text{free}}(c_j), V_{t+1}(c_j) \leftarrow \text{RKf}(c_j, \hat{\xi}_t(c_j), y_t, V_t(c_j), \{A, B, C, D\})$ 
4:    $\triangleright$  MPC solver with nominal model
5:    $u_t(c_j), \hat{\xi}_{t+1}(c_j), J_t(c_j) \leftarrow \text{MPC}(\hat{\xi}_{t+1, \text{free}}(c_j), \{A, B, C, D\}, N)$ 
6:    $\triangleright$  Add past prediction errors to the controller cost
7:    $J_t(c_j) \leftarrow J_t(c_j) + \sum_{i=-L}^0 \beta^i \|y_i - \hat{y}_{i|i-1}(c_j)\|^2$ 
8:  $\triangleright$  Optimization step, problem (5)
9:  $c_{\text{best}} \leftarrow \arg \min_{c_j} J_t(c_j)$ 
10:  $\triangleright$  Apply input
11:  $u_t \leftarrow u_t(c_{\text{best}})$ 

```

Algorithm 4 Robust Kalman filter at time t .

Require: $c_t, \hat{x}_t, y_t, V_t$,**Ensure:** $\hat{x}_{t+1}, V_{t+1}, G_t, \lambda_t$

- 1: $G_t \leftarrow (A_t V_t C_t^\top + B_t D_t^\top)^\top (C_t V_t C_t^\top + D_t D_t^\top)^{-1}$
 - 2: $\hat{x}_{t+1} \leftarrow A_t \hat{x}_t + G_t (y_t - C_t \hat{x}_t)$
 - 3: $P_{t+1} \leftarrow (A_t - G_t C_t) V_t (A_t - G_t C_t)^\top + (B_t - G_t D_t)(B_t - G_t D_t)^\top$
 - 4: Find λ_t such that $c_t = \gamma_\tau(P_{t+1}, \lambda_t)$
 - 5: Compute
 $V_{t+1} \leftarrow (I_n - \lambda_t^{-1} P_{t+1})^{-1} P_{t+1}$
-

Given that it is evaluated repeatedly throughout the algorithm, and that subsequent computations depending on its value exhibit high sensitivity, both numerical stability and computational efficiency must be considered when determining λ_t (step 4 of Algorithm 4). In this work, we employ a variant of the regula falsi secant method, incorporating the modifications proposed by Anderson and Björk in [2] (a general pseudo-code for this family of algorithms can be found in [3]).

Algorithm 5 Least-Favorable Model for the time interval $[k, k + N]$.

Require: $c, V_k, N, N_f > N$ (default $N_f = 2N$)**Ensure:** $\tilde{A}_t, \tilde{B}_t, \tilde{C}_t, \tilde{D}_t$ for $t \in [k, k + N]$

- 1: $\triangleright$ *Forward Riccati iterations at least to large enough time N_f* $\triangleleft$
 - 2: **for** $t = 1, \dots, N_f$ **do**
 - 3: $\quad V_{t+1}, G_t, \lambda_t \leftarrow RKF(c, V_t)$ $\triangleright$ *Algorithm 4 without state-prediction (step 2)*
 - 4: $\triangleright$ *Backward sweep to compute LFM covariances* $\triangleleft$
 - 5: $W_{N_f}^{-1} \leftarrow \lambda_{N_f}^{-1} I_n$
 - 6: **for** $t = N_f - 1, \dots, 1$ **do**
 - 7: $\quad \triangleright$ *temporary matrices* $\triangleleft$
 - 8: $\quad A_{tmp} \leftarrow A_t - G_t C, \quad B_{tmp} \leftarrow B_t - G_t D$
 - 9: $\quad \triangleright$ *computation of K^{-1}, H, L* $\triangleleft$
 - 10: $\quad K_t^{-1} \leftarrow I_{n+p} - B_{tmp}^\top W_t^{-1} B_{tmp}$
 - 11: $\quad H_t \leftarrow K_t B_{tmp}^\top W_t^{-1} A_{tmp}$
 - 12: $\quad L_t \leftarrow \text{cholesky}(K^{-1})^{-1}$
 - 13: $\quad \triangleright$ *update values for next iteration* $\triangleleft$
 - 14: **if** $t = 1$ **then**
 - 15: $\quad \text{exit cycle}$
 - 16: **else**
 - 17: $\quad \Omega_t^{-1} \leftarrow A_{tmp}^\top W_{t+1} A_{tmp} + H_t^\top K_t^{-1} H_t$
 - 18: $\quad W_t^{-1} \leftarrow \Omega_t^{-1} + \lambda_{t-1}^{-1} I_n$
 - 19: $\triangleright$ *Forward sweep to compute the LFM matrices to N* $\triangleleft$
 - 20: **for** $t = 1, \dots, N$ **do**
 - 21: $\quad A_{tmp} \leftarrow A_t - G_t C, \quad B_{tmp} \leftarrow B_t - G_t D$
 - 22: $\quad$ Compute the matrices of LFM:
 $\quad \tilde{A}_t \leftarrow \begin{bmatrix} A_t & B_t H_t \\ 0 & A_{tmp} + B_{tmp} H_t \end{bmatrix}, \quad \tilde{B}_t \leftarrow \begin{bmatrix} B_t \\ B_{tmp} \end{bmatrix} L_t, \quad \tilde{C}_t \leftarrow [C_t \quad D H_t], \quad \tilde{D}_t \leftarrow D_t L_t$
-

References

- [1] B. C. Levy and R. Nikoukhah, “Robust state space filtering under incremental model perturbations subject to a relative entropy tolerance,” *IEEE Trans. Autom. Control*, vol. 58, no. 3, pp. 682–695, Mar. 2013.
- [2] N. Anderson and Å. Björck, “A new high order method of regula falsi type for computing a root of an equation,” *BIT Numerical Mathematics*, vol. 13, no. 3, pp. 253–264, 1973.
- [3] S. M. L. Galdino, “A family of regula falsi root-finding methods,” 2011. [Online]. Available: <https://api.semanticscholar.org/CorpusID:15854811>.